

Lecture 5 - Object Design: Case Study Presentations in Week 6
Topic: Responsibility-Driven Design (RDD), CRC Cards, Cohesion, Coupling, and the Towers of Hanoi
Format: Each Group: 5-10 Minute Live Presentation & Discussion
Architectural Strike Teams: The Towers of Hanoi Project

Background Context for All Groups:

"In the jungle outside Hanoi, monks are moving 64 rings stacked on 3 poles. A ring may only be moved one at a time, and a larger ring cannot be placed on top of a smaller ring. When all rings are moved from Pole 1 to Pole 3, the world will end."

Your software firm has been hired to build an object-oriented simulation engine for this scenario. Break into your assigned "Strike Teams" below to design the architecture.

Group 1: The Object Discovery Team

Focus: Noun-Phrase Analysis & Identifying Candidate Classes (*Refers to Tutorial Q1*)

Scenario: You are starting with a blank whiteboard. Before writing any CRC cards, you must define the "Cast of Characters" (the Classes) that will make up this biological community of objects.

Presentation Prompts:

1. **Noun Analysis:** Perform a quick noun-phrase analysis on the problem description. List the legitimate "Candidate Classes" you will build in code.

=> Student: Showcases the university students and contains attributes including StudentID, FullName, CardUID (for rapid swiping), and Department.

=> Lecturer: Represents the academic staff monitoring the session, containing LecturerID, FullName, and Department.

=> Session: Emphasizes a specific class instance in a timeslot include SessionID, CourseCode, RoomNumber, Date, StartTime, and EndTime.

=> AttendanceRecord: The transactional entity bridging Students and Sessions (RecordID, StudentID, SessionID, SwipeTimestamp, and AttendanceStatus Present/Late/Absent).

=> AccessDevice (Future-Proof): Represents the electronic door locks. Contains DeviceID, RoomNumber, and IsLocked

2. **Discard Pile:** Which nouns from the text did you aggressively discard and why? (Give an example of an external actor, an irrelevant concept, and a simple attribute).

=> External Factor => The University/Swinburne

=> "The University" is the macro-environment hosting the system, not an entity *inside* the software boundaries that we need to instantiate or compute logic for.

3. **Roles:** Pick one of your core classes (e.g., the Pole). What is its Role? Is it a Controller, a Coordinator, or an Information Holder?

=> ClassSession:

=> Its Role: Primarily an Information Holder, acting secondarily as a Coordinator.

=> Detailed Rationale:

- As an *Information Holder*, **ClassSession** maintains the structural knowledge of a class instance (CourseCode, StartTime, EndTime, RoomNumber).
- As a *Coordinator*, it cluster-manages the collection of active **AttendanceRecords** generated during its specific timeslot window. It encapsulates the domain logic required to determine whether an incoming swipe timestamp is valid, late, or completely outside the permissible grace period for that session.

Group 2: The Responsibility Architects

Focus: Doing vs. Knowing & The Information Expert Principle (*Refers to Tutorial Q2 & Q3*)

Scenario: You have your blank CRC cards for your core classes. Now you need to assign responsibilities (obligations) without creating a centralized "God Class."

Presentation Prompts:

1. **Doing vs. Knowing:** Identify two "Knowing" responsibilities and two "Doing" responsibilities in this system. Which specific classes get assigned these responsibilities?

=> 'Doing' Responsibilities:

- Knowing student eligibility and card mapping: Assigned to the Student (or SmartCard) class. It is obligated to know its own CardUID and profile details.
- Knowing the current state of a class session: Assigned to the ClassSession class. It must know its scheduled StartTime, EndTime, and the list of checked-in records.

=> 'Knowing' Responsibilities:

- Validating and recording a single check-in transaction: Assigned to the AttendanceRecord class. Upon instantiation by a card swipe, it executes the action of checking its timestamp against the session's grace period.
- Broadcasting lock-control signals (Future-Proofing): Assigned to the AccessDevice class. It is responsible for physically communicating with the hardware layer to change the state of the door lock when triggered.

2. The Information Expert: A core rule of Hanoi is "a larger ring cannot be placed on a smaller ring." Applying the Information Expert principle, which class should be responsible for validating if a move is legal, and why?

=> The Assignment: The Pole class should be responsible for validating whether a ring placement move is legal.

=> The Rationale: To determine if a move is legal, the system needs to know the size of the top ring currently sitting on the destination pole, and compare it to the size of the incoming ring.

3. The Setup Problem: The tutorial hints that you need a new class responsible for setting up the poles and starting the game. Why is it bad Object-Oriented Design to force the Ring or Pole class to build the whole game themselves?

=> Violation of the Single Responsibility Principle (SRP): A Pole object exists to hold and manage its own rings. If you force it to know how many poles are in a game, how to instantiate them, and how to distribute the initial rings, it suddenly has two entirely distinct reasons to change: managing its own structure vs. managing the global game rules.

=> Unnecessary High Coupling: If a Pole is responsible for building the game, it must explicitly know about the creation logic of every other class, including the user interface, game settings, and other instances. It becomes impossible to reuse the Pole class in any other context or alternative version of the game.

=> High Cohesion Breakdown: Objects should look after their own local business. Global configuration, initialization, and game setup workflows are architectural structural concerns. They should be delegated to a dedicated Creational Pattern/Coordinator class (such as a GameInitializer, GameSuite, or Factory), ensuring domain components remain highly cohesive, lightweight, and easy to maintain.

Group 3: The Message-Passing Role-Players

Focus: Collaborations & "Tell, Don't Ask" (*Refers to Tutorial Q4*)

Scenario: Alan Kay said OOP is all about biological cells passing messages. You need to map out the exact sequence of messages (collaborations) required to successfully move a ring.

Presentation Prompts:

1. **Client-Server Contract:** When moving a ring from Pole 1 to Pole 3, define the Client Server relationship. Who is the Client making the request, and who is the Server providing the service?

=> The Client: The GameController (or HanoiGame coordinator class). It acts as the Client because it intercepts the user's intent to make a move, triggers the workflow, and holds the high-level obligation to advance the game state.

=> The Server: The Pole class. Specifically, Pole 1 acts as a Server providing a "*pop/remove top ring*" service, and Pole 3 acts as a Server providing an "*add incoming ring*" service.

2. **The Walkthrough:** Trace the exact sequence of messages passed between objects to successfully move the top ring from Pole 1 to Pole.

- GameController => Pole 3 (canAccept(topRingOfPole1)): The controller asks the destination pole if it can legally accept the ring currently sitting on top of Pole 1.
- Pole 3 => Self-Validation: Pole 3 checks its internal collection. If it is empty or if its current top ring is larger than the incoming ring, it returns true.
- GameController => Pole 1 (removeTopRing()): Once the move is validated as legal, the controller instructs Pole 1 to detach and return its top ring object.
- Pole 1 => GameController (Returns Ring object): Pole 1 updates its internal stack and hands the physical Ring instance back to the controller.
- GameController => Pole 3 (pushRing(Ring)): The controller passes the acquired Ring object to Pole 3. Pole 3 appends the ring to its internal stack, completing the collaboration.

3. **Tell, Don't Ask:** How do you move the ring *without* violating encapsulation? Explain how "Tell, Don't Ask" prevents you from just extracting the ring's size, checking the pole's array yourself, and forcing the data in.

=> The "Tell, Don't Ask" principle states that you should tell objects what action to perform rather than asking them for their internal data, manipulating that data outside the object, and forcing it back in.

- The Bad, Non-OO Way (Asking): The controller grabs Pole 1's raw array of rings, extracts the last element, grabs Pole 3's raw array, manually runs an if (size1 < size3) condition in the controller logic, and directly modifies Pole 3's array elements. This completely ruins encapsulation, exposes the inner database/array structure of the poles, and creates a fragile, highly-coupled system.
- The Clean OO Way (Telling): We respect the boundary of the object. We tell Pole 3 to evaluate the legality via a clean interface (pole3.canAccept(incomingRing)), and then we tell Pole 3 to add it (pole3.pushRing(incomingRing)).

=> By "telling" the pole what to do instead of "asking" it for its structural arrays, the internal implementation of how a pole stores

rings (whether it uses a List, a Stack, or an Array) remains entirely hidden from the rest of the software. We can rewrite the internal data structure of the Pole class tomorrow, and the rest of our system won't break.

Group 4: The Refactoring Analysts

Focus: Cohesion, Coupling, & Anti-Patterns (*Refers to Tutorial Q5*)

Scenario: A junior developer handed you their Hanoi design. They created a single Monk class that holds a giant 2D array of all rings and contains a massive 500-line calculateMove() method. Their Ring and Pole classes contain absolutely no methods, only variables with getters and setters.

Presentation Prompts:

1. **Evaluating Metrics:** Evaluate this junior developer's design using the concepts of *Cohesion* and *Coupling*. Is the cohesion high or low? Is the coupling tight or loose?

=> Cohesion: Low. The Monk class has too many responsibilities. It manages game rules, handles initialization, tracks the complete state of all physical objects via the 2D array, and executes movement calculations.

=> Coupling: Tight. Because Ring and Pole contain only raw data, the Monk class must directly access and rely on their internal structures. Any change to how a ring or pole holds data instantly breaks the 500-line calculateMove() method.

2. **Identifying Anti-Patterns:** What two specific OO anti-patterns has the junior developer created by designing the system this way? (Hint: Think about "God Classes" and "Anemic Domain Models").

=> The God Class (or Blob): Represented by the Monk class. It concentrates all system intelligence, logic, and control into a single, massive 500-line method, reducing the rest of the application to data containers.

=> Anemic Domain Model: Represented by the Ring and Pole classes. They are hollow structures containing data (getters/setters) but zero operational behavior or domain rules, violating fundamental object-oriented principles.

3. **The Fix:** How do you redesign this to achieve "evenly distributed system intelligence" across the CRC cards?

=> Dismantle the 2D Array: Remove the global grid. Let each individual Pole class maintain its own internal Stack<Ring> collection.

=> Move Validation to the Expert: Shift the validation logic out of calculateMove(). Give the Pole class a canAccept(Ring) method so it evaluates its own placement rules.

=> Move Action to the Target: Give the Pole class push(Ring) and pop() methods to manage its own state changes.

=> Demote the God Class: Refactor the Monk class into a lightweight HanoiGame coordinator. Its only job should be orchestrating messages between the poles (e.g., telling Pole 1 to pop and telling Pole 3 to push).